

Relatorio do trial

PARTICIPANTE

Luis Henrique Gonçalves Barbosa

KATA

Tarifador de Logística Urbana

TRATAMENTO

Sem assistente de IA

TEMPO FINAL

18:50

TEMPO ESGOTADO

Nao

INICIO

16/09/2026, 14:05:14

FIM

16/09/2026, 14:24:04

RESULTADO E OBSERVACOES

8 de 8 testes unitários de aceitação passaram com sucesso (100%). Codificação manual.

ARQUIVOS ANEXADOS

4 arquivo(s) de código ou texto anexado(s) ao trial.

- texto-do-trial.txt (164 linhas)
- TarifadorLogistica.java (112 linhas)
- ModalidadeEntrega.java (18 linhas)
- TipoCarga.java (20 linhas)

texto-do-trial.txt

Participante: Luis Henrique Gonçalves Barbosa
Kata: Tarifador de Logística Urbana
Tratamento: SEM_IA
Tempo: 18:50

Código implementado:

```
// === ModalidadeEntrega.java ===
package br.ufc.lab02.logistica;

public enum ModalidadeEntrega {
    PADRAO(1.00),
    EXPRESSA(1.40),
    MESMO_DIA(1.80);

    private final double fatorMultiplicador;

    ModalidadeEntrega(double fatorMultiplicador) {
        this.fatorMultiplicador = fatorMultiplicador;
    }

    public double getFatorMultiplicador() {
        return fatorMultiplicador;
    }
}

// === TarifadorLogistica.java ===
package br.ufc.lab02.logistica;

import java.math.BigDecimal;
import java.math.RoundingMode;

public class TarifadorLogistica {

    private static final BigDecimal TARIFA_BASE_CURTA = BigDecimal.valueOf(10.00);
    private static final BigDecimal TARIFA_BASE_MEDIA = BigDecimal.valueOf(25.00);
    private static final BigDecimal VALOR_KM_EXCEDENTE = BigDecimal.valueOf(1.50);
    private static final BigDecimal VALOR_KG_EXCEDENTE = BigDecimal.valueOf(3.00);
    private static final BigDecimal FATOR_DESCONTO_FIDELIDADE = BigDecimal.valueOf(0.90);

    public BigDecimal calcularFrete(
        double distanciaKm,
        double pesoKg,
        ModalidadeEntrega modalidade,
        TipoCarga tipoCarga,
        boolean clienteFidelidade) {

        validarDadosDaEntrega(distanciaKm, pesoKg, modalidade, tipoCarga);

        BigDecimal custoDaDistancia = calcularCustoDaDistancia(distanciaKm);
        BigDecimal custoDoPeso = calcularCustoDoPeso(pesoKg);
        BigDecimal subtotalDoTransporte = custoDaDistancia.add(custoDoPeso);
        BigDecimal totalComModalidade = aplicarModalidade(subtotalDoTransporte,
modalidade);
        BigDecimal totalComManuseio = adicionarTaxaDeManuseio(totalComModalidade,
tipoCarga);
        BigDecimal totalComFidelidade = aplicarDescontoDeFidelidade(
```

```

        totalComManuseio,
        clienteFidelidade
    );

    return formatarValor(totalComFidelidade);
}

private void validarDadosDaEntrega(
    double distanciaKm,
    double pesoKg,
    ModalidadeEntrega modalidade,
    TipoCarga tipoCarga) {

    if (distanciaKm <= 0) {
        throw new IllegalArgumentException("Distancia deve ser maior que zero.");
    }

    if (pesoKg <= 0) {
        throw new IllegalArgumentException("Peso deve ser maior que zero e de no
maximo 50 kg.");
    }

    if (pesoKg > 50.0) {
        throw new IllegalArgumentException("Peso deve ser maior que zero e de no
maximo 50 kg.");
    }

    if (modalidade == null) {
        throw new IllegalArgumentException("Modalidade e tipo de carga sao
obrigatorios.");
    }

    if (tipoCarga == null) {
        throw new IllegalArgumentException("Modalidade e tipo de carga sao
obrigatorios.");
    }
}

private BigDecimal calcularCustoDaDistancia(double distanciaKm) {
    if (distanciaKm <= 15.0) {
        return TARIFA_BASE_CURTA;
    }

    if (distanciaKm <= 50.0) {
        return TARIFA_BASE_MEDIA;
    }

    double quilometrosExcedentes = distanciaKm - 50.0;
    BigDecimal quantidadeExcedente = BigDecimal.valueOf(quilometrosExcedentes);
    BigDecimal custoDoExcedente = VALOR_KM_EXCEDENTE.multiply(quantidadeExcedente);

    return TARIFA_BASE_MEDIA.add(custoDoExcedente);
}

private BigDecimal calcularCustoDoPeso(double pesoKg) {
    if (pesoKg <= 5.0) {
        return BigDecimal.ZERO;
    }

    double quilogramasExcedentes = pesoKg - 5.0;
    BigDecimal quantidadeExcedente = BigDecimal.valueOf(kilogramasExcedentes);

```

```

        return VALOR_KG_EXCEDENTE.multiply(quantidadeExcedente);
    }

    private BigDecimal aplicarModalidade(BigDecimal subtotal, ModalidadeEntrega
modalidade) {
        BigDecimal fatorDaModalidade =
BigDecimal.valueOf(modalidade.getFatorMultiplicador());
        return subtotal.multiply(fatorDaModalidade);
    }

    private BigDecimal adicionarTaxaDeManuseio(BigDecimal subtotal, TipoCarga tipoCarga) {
        BigDecimal taxaDeManuseio = tipoCarga.getTaxaManuseio();
        return subtotal.add(taxaDeManuseio);
    }

    private BigDecimal aplicarDescontoDeFidelidade(BigDecimal valor, boolean
clienteFidelidade) {
        if (clienteFidelidade) {
            return valor.multiply(FATOR_DESCONTO_FIDELIDADE);
        }

        return valor;
    }

    private BigDecimal formatarValor(BigDecimal valor) {
        return valor.setScale(2, RoundingMode.HALF_EVEN);
    }
}

```

```

// === TipoCarga.java ===
package br.ufc.lab02.logistica;

import java.math.BigDecimal;

public enum TipoCarga {
    NORMAL(BigDecimal.valueOf(0.00)),
    FRAGIL(BigDecimal.valueOf(15.00)),
    PERECIVEL(BigDecimal.valueOf(25.00));

    private final BigDecimal taxaManuseio;

    TipoCarga(BigDecimal taxaManuseio) {
        this.taxaManuseio = taxaManuseio;
    }

    public BigDecimal getTaxaManuseio() {
        return taxaManuseio;
    }
}

```

TarifadorLogistica.java

```
package br.ufc.lab02.logistica;

import java.math.BigDecimal;
import java.math.RoundingMode;

public class TarifadorLogistica {

    private static final BigDecimal TARIFA_BASE_CURTA = BigDecimal.valueOf(10.00);
    private static final BigDecimal TARIFA_BASE_MEDIA = BigDecimal.valueOf(25.00);
    private static final BigDecimal VALOR_KM_EXCEDENTE = BigDecimal.valueOf(1.50);
    private static final BigDecimal VALOR_KG_EXCEDENTE = BigDecimal.valueOf(3.00);
    private static final BigDecimal FATOR_DESCONTO_FIDELIDADE = BigDecimal.valueOf(0.90);

    public BigDecimal calcularFrete(
        double distanciaKm,
        double pesoKg,
        ModalidadeEntrega modalidade,
        TipoCarga tipoCarga,
        boolean clienteFidelidade) {

        validarDadosDaEntrega(distanciaKm, pesoKg, modalidade, tipoCarga);

        BigDecimal custoDaDistancia = calcularCustoDaDistancia(distanciaKm);
        BigDecimal custoDoPeso = calcularCustoDoPeso(pesoKg);
        BigDecimal subtotalDoTransporte = custoDaDistancia.add(custoDoPeso);
        BigDecimal totalComModalidade = aplicarModalidade(subtotalDoTransporte,
modalidade);
        BigDecimal totalComManuseio = adicionarTaxaDeManuseio(totalComModalidade,
tipoCarga);
        BigDecimal totalComFidelidade = aplicarDescontoDeFidelidade(
            totalComManuseio,
            clienteFidelidade
        );

        return formatarValor(totalComFidelidade);
    }

    private void validarDadosDaEntrega(
        double distanciaKm,
        double pesoKg,
        ModalidadeEntrega modalidade,
        TipoCarga tipoCarga) {

        if (distanciaKm <= 0) {
            throw new IllegalArgumentException("Distancia deve ser maior que zero.");
        }

        if (pesoKg <= 0) {
            throw new IllegalArgumentException("Peso deve ser maior que zero e de no
maximo 50 kg.");
        }

        if (pesoKg > 50.0) {
            throw new IllegalArgumentException("Peso deve ser maior que zero e de no
maximo 50 kg.");
        }

        if (modalidade == null) {

```

```

        throw new IllegalArgumentException("Modalidade e tipo de carga sao
obrigatorios.");
    }

    if (tipoCarga == null) {
        throw new IllegalArgumentException("Modalidade e tipo de carga sao
obrigatorios.");
    }
}

private BigDecimal calcularCustoDaDistancia(double distanciaKm) {
    if (distanciaKm <= 15.0) {
        return TARIFA_BASE_CURTA;
    }

    if (distanciaKm <= 50.0) {
        return TARIFA_BASE_MEDIA;
    }

    double quilometrosExcedentes = distanciaKm - 50.0;
    BigDecimal quantidadeExcedente = BigDecimal.valueOf(quilometrosExcedentes);
    BigDecimal custoDoExcedente = VALOR_KM_EXCEDENTE.multiply(quantidadeExcedente);

    return TARIFA_BASE_MEDIA.add(custoDoExcedente);
}

private BigDecimal calcularCustoDoPeso(double pesoKg) {
    if (pesoKg <= 5.0) {
        return BigDecimal.ZERO;
    }

    double quilogramasExcedentes = pesoKg - 5.0;
    BigDecimal quantidadeExcedente = BigDecimal.valueOf(kilogramasExcedentes);

    return VALOR_KG_EXCEDENTE.multiply(quantidadeExcedente);
}

private BigDecimal aplicarModalidade(BigDecimal subtotal, ModalidadeEntrega
modalidade) {
    BigDecimal fatorDaModalidade =
BigDecimal.valueOf(modalidade.getFatorMultiplicador());
    return subtotal.multiply(fatorDaModalidade);
}

private BigDecimal adicionarTaxaDeManuseio(BigDecimal subtotal, TipoCarga tipoCarga) {
    BigDecimal taxaDeManuseio = tipoCarga.getTaxaManuseio();
    return subtotal.add(taxaDeManuseio);
}

private BigDecimal aplicarDescontoDeFidelidade(BigDecimal valor, boolean
clienteFidelidade) {
    if (clienteFidelidade) {
        return valor.multiply(FATOR_DESCONTO_FIDELIDADE);
    }

    return valor;
}

private BigDecimal formatarValor(BigDecimal valor) {
    return valor.setScale(2, RoundingMode.HALF_EVEN);
}

```

}

ModalidadeEntrega.java

```
package br.ufc.lab02.logistica;

public enum ModalidadeEntrega {
    PADRAO(1.00),
    EXPRESSA(1.40),
    MESMO_DIA(1.80);

    private final double fatorMultiplicador;

    ModalidadeEntrega(double fatorMultiplicador) {
        this.fatorMultiplicador = fatorMultiplicador;
    }

    public double getFatorMultiplicador() {
        return fatorMultiplicador;
    }
}
```

TipoCarga.java

```
package br.ufc.lab02.logistica;

import java.math.BigDecimal;

public enum TipoCarga {
    NORMAL(BigDecimal.valueOf(0.00)),
    FRAGIL(BigDecimal.valueOf(15.00)),
    PERECIVEL(BigDecimal.valueOf(25.00));

    private final BigDecimal taxaManuseio;

    TipoCarga(BigDecimal taxaManuseio) {
        this.taxaManuseio = taxaManuseio;
    }

    public BigDecimal getTaxaManuseio() {
        return taxaManuseio;
    }
}
```